Received XX Month 2026, accepted XX Month 2026, date of publication XX Month 2026, date of current version XX Month 2026.
Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

Learned Versus Rule-Based Energy Management for Electric Vehicles on a Microcontroller: A Processor-in-the-Loop Study With Pre-Registered Acceptance Gates

MANIKANTA GONUGONDLA[1] AND [CO-AUTHOR NAMES TO BE CONFIRMED][1]

1[Department and institution to be confirmed]

Corresponding author: Manikanta Gonugondla (e-mail to be confirmed).

MOCK MANUSCRIPT prepared for internal review. Not submitted to any journal. Dates, volume and Digital Object Identifier are placeholders.

ABSTRACT Learned energy managers for electric vehicles are usually evaluated in simulation on a workstation, with the vehicle model and the baseline taken on trust. This paper reports a processor-in-the-loop study in which a rule-based energy manager, a Soft Actor-Critic policy and a power forecaster run on an ESP32-S3 microcontroller at 100 Hz against a longitudinal vehicle model identified from logged trips. Acceptance gates and their pass criteria were fixed before each experiment, the one criterion revised afterwards is reported, and every result is labelled as measured on the target or simulated on the host. The main result is negative. Over 36 runs and 2,282,832 control steps with no late step, the learned policy used less energy than the rules in 1 of 16 settings and 0.94% more on average, and tracked speed worse on the aggressive US06 cycle. An earlier apparent gain of 8.28% was traced to a power limit in the vehicle model that capped billed energy without reducing delivered torque, and is withdrawn. A Transformer forecaster improved on persistence by 16.6% at a 3 s horizon and ran in 36.46 ms of a 40 ms budget, yet removing the forecast from the policy changed nothing measurable. The gains came from estimation instead. A two-state Kalman speed estimator cut the energy lost at ten times the sensor-noise floor from 9.44 to 0.24 Wh/km, and a bias-aware detector caught 120 of 120 injected speed-sensor faults with no false alarm in one hour of clean driving. Two acceptance gates fail and are reported as failures.

INDEX TERMS Electric vehicles, energy management, fault detection, Kalman filters, microcontrollers, processor-in-the-loop, reinforcement learning, time series forecasting.

I. INTRODUCTION

A battery electric vehicle (EV) spends its stored energy on traction, on auxiliaries such as cabin heating, and on losses that depend on how the vehicle is driven. A supervisory energy manager decides, at every control step, how much of the available battery power the drivetrain and the heater may take. A forecast of battery power a few seconds ahead can give that decision some look-ahead, and energy management for electrified powertrains has a long record in the hybrid-vehicle literature [1].

Data-driven models now estimate range, consumption and driving parameters from vehicle data [2], [3]. Whether such a model helps inside a controller is not settled by prediction accuracy alone.

The starting point of this work is the study by Tangi et al. [4]. That study predicted EV range, optimum acceleration and optimum velocity with Random Forest, Extra Trees, Linear Regression and Long Short-Term Memory (LSTM) models and their ensembles. It used a synthetically generated dataset of 2,000 samples, averaged results over ten randomised partitions, and served the trained models through a web application. The full ensemble gave the most accurate predictions. The study showed that data-driven models can estimate driving parameters, and its authors named real-world data and embedded evaluation among the next steps. This paper takes up three questions that follow from there. Do the conclusions hold on logged driving data rather than synthetic data? What happens when the predictions and the policies built on them close a control loop around a vehicle? And can the whole chain run on an embedded controller under a real-time deadline?

The wider literature leaves the same questions open. Learned energy managers, from deep Q-network designs to Soft Actor-Critic (SAC) policies [5], are often reported from simulation [6], [7], and a review of the field is available [8]. Hardware-in-the-loop (HIL) validation exists [9], yet among the works compared in Table 1 none states a loop timing measured on the target processor. Deep reinforcement learning

results also vary with the random seed and with how the baseline is tuned [10], and the problems of deploying such controllers on real systems are well documented [11]. A convincing comparison therefore has to run on the target with its timing measured, against a baseline that has not been left weak, and on a vehicle model that has itself been audited.

This work builds that comparison as a processor-in-the-loop (PIL) study. The controller runs on an ESP32-S3 microcontroller [12] at 100 Hz, while a longitudinal vehicle model runs on a host computer and exchanges state and commands with it every 10 ms. The arrangement is PIL and not HIL: the processor is real, but the motor, inverter and battery are simulated. The vehicle model is identified and validated on logged BMW i3 trips [13], 33 of 70 of which passed the eligibility tests, and checked against the Vehicle Energy Dataset (VED) [14], and the controllers run the UDDS, HWFET and US06 drive cycles [15]. Acceptance gates and pass criteria were written before each experiment, results that miss a criterion are reported as misses, and every result is labelled measured if it ran on the microcontroller or simulated if it ran on the host only.

The main finding is negative: once a fault in the vehicle model had been found and corrected, a learned SAC policy did not beat a rule-based energy manager. It used less energy in 1 of 16 valid settings on the target, 0.94% more energy on average, and tracked speed worse on the aggressive US06 cycle (measured). Giving the policy the power forecast changed nothing measurable. An apparent gain of 8.28% in an earlier campaign came from the model fault and is withdrawn in this paper. The energy lost under sensor noise turned out to sit in the speed loop, below the energy manager, and a two-state Kalman speed estimator [16] cut the loss from 9.44 to 0.24 Wh/km at ten times the nominal noise on US06 (measured). A bias-aware detector caught 120 of 120 injected speed-sensor faults with no false alarm in one clean hour (measured), while learned reviewers classified faults better offline and controlled the vehicle worse in the loop (simulated). A dynamic-programming speed planner saved 10.4% of energy on a US06-derived route without traffic and 5.1% when held within 25 m of a recorded drive (simulated).

This paper makes four contributions.

- A PIL testbed on a microcontroller at 100 Hz, with acceptance gates fixed before each experiment, the one revised criterion reported, and loop timing measured.
- A measured comparison of a rule-based energy manager with an SAC policy over 16 settings on the target, including the withdrawal of an earlier apparent gain traced to a vehicle-model fault.
- A comparison of forecasting models by accuracy against on-device cost, with the Transformer's latency measured on the target, in which linear regression gives 87% of the Transformer's benefit for 1.2% of its estimated compute.
- A two-state Kalman speed estimator and a bias-aware sensor-fault detector, both measured on the target, compared with learned reviewers.

Section II reviews related work. Section III describes the platform, the acceptance gates and the labelling of results. Section IV presents the vehicle model and the two datasets. Section V covers on-device power forecasting, and Section VI compares the rule-based manager with the learned policy. Section VII treats speed estimation and fault detection, Section VIII the simulated speed planner, and Section IX the limits of the evidence. Section X concludes.

II. RELATED WORK

A. DATA-DRIVEN RANGE AND ENERGY PREDICTION

Machine-learning models of EV energy use are usually fitted to logged or generated trips and evaluated offline. The study of Tangi et al. [4] belongs here: it compared four model families and their ensembles on a synthetic dataset and exposed them through a web application. Zhang et al. [2] worked from real-world driving data of 55 electric taxis in Beijing and reported a root mean squared error (RMSE) of 0.159 kWh with a prediction framework that includes driving-condition prediction. Baluch et al. [3] fitted a nonlinear autoregressive model with exogenous inputs to on-road data from four trips and reported a mean squared error 30% below that of an LSTM network, with fewer parameters. Two public logged sources exist for this kind of work: the Vehicle Energy Dataset [14], a large fleet collection, and the BMW i3 trips [13], which carry battery and heating data. Architectures such as the Transformer [17] and training with physics-based loss terms [18] are candidates for such forecasts. The three prediction studies above predict without acting on the prediction, and none of them states a measured run time on a vehicle-class processor.

B. REINFORCEMENT LEARNING FOR ENERGY MANAGEMENT

Reinforcement learning has been applied to energy management mainly in hybrid vehicles. A deep Q-network strategy for a power-split hybrid bus reached about 89% of the fuel economy of dynamic programming on an unseen driving condition [6]. A SAC-based strategy for a hybrid tracked vehicle was compared with standard SAC and with DDPG in simulation [7], and a reinforcement-learning power manager for a battery and ultracapacitor system in an all-climate electric vehicle was compared with rules and dynamic programming [9]. Reviews cover the field and its open problems [1], [8]. The reference points in this literature are

classical. Dynamic programming [19] gives the optimum for a known cycle, Pontryagin's minimum principle offers an analytical route to the same problem [20], and the equivalent consumption minimisation strategy (ECMS) turns the problem into an instantaneous cost [21], [22]. Rule-based strategies are the usual deployable baseline. Results of deep reinforcement learning differ strongly across seeds and implementation choices [10], so a gain over a weakly tuned rule set says little about the method.

C. EMBEDDED AND IN-THE-LOOP VALIDATION

Vehicle controllers are normally validated through a staged path of model-, software-, hardware-in-the-loop and vehicle tests [1]. HIL benches drive a real controller or real energy storage from a simulated vehicle, as in the battery and ultracapacitor in-the-loop validation of a reinforcement-learning power manager on an xPC Target platform [9]. PIL occupies an earlier step in which the target processor executes the code while the plant stays simulated. On the machine-learning side, inference frameworks for microcontrollers [23] and joint model and library design [24] show that small devices can run networks, and integer quantisation [25] and knowledge distillation [26] are the standard routes to smaller models. Dulac-Arnold et al. [11] list the constraints that separate laboratory reinforcement learning from deployment, among them real-time inference. What the cited works do not give, in the vehicle-energy setting, is a measured loop with a stated deadline on the target together with a rule baseline run through the same loop.

D. ESTIMATION AND SENSOR-FAULT DETECTION

Kalman filtering [16] is a standard tool for state estimation, including vehicle speed; a recent example is an extended Kalman filter that uses inertial, steering-angle and wheel-speed sensors [27]. Model-based fault detection compares measured signals with model predictions and examines the residual [28]. Learned detectors are the alternative: LSTM networks have been applied to anomaly detection in time series [29], a survey covers anomaly detection in general [30], and gated recurrent units [31] and FastGRNN, a gated recurrent network designed to be kilobytes in size [32], are the small recurrent models considered for devices of this class. The comparison made here is between a classical estimator with a model-based detector and learned reviewers, with every candidate run in the vehicle loop as well as scored on labelled faults.

Table 1 places six prior works and this study along the axes that matter here. A cell is filled only where the source states the point.

III. EXPERIMENTAL PLATFORM AND METHOD

Every result in the paper carries one of two labels. measured means the quantity was recorded while the code ran on the microcontroller; simulated means it came from the host alone.

A. PROCESSOR-IN-THE-LOOP ARRANGEMENT

The arrangement is processor-in-the-loop (PIL), not hardware-in-the-loop. The processor is real: an ESP32-S3 runs the control firmware, receives its inputs over a serial link and must answer before a wall-clock deadline [12]. The vehicle, motor, inverter, battery, driver and sensors are simulated on a host laptop. Faults are injected in software.

Fig. 1 shows one control period. The target is the clock master. Every 10 ms it stamps and sends a step request. The host builds a measurement from the plant state, adds seeded sensor noise to the measured channels and replies with a state frame of 11 floating-point values. The target runs its control code and returns an actuation frame of four values: torque, regeneration blend, power limit and auxiliary derate. The host advances the vehicle model by 10 ms with them and logs the outcome.

The firmware can run in a synchronous mode, which blocks inside the slot until the reply arrives, or in a pipelined mode, which requests the next state, acts on the latest one already held and reads replies for at most 1 ms. The pipelined mode is the default and the one reported here. It costs one period of age on the state used at each step, so the controller acts on a 10 ms old measurement.

TABLE 1. Positioning of six prior studies and this work; cells marked not stated could not be confirmed from the source.

Work	Data	Closed loop	On target processor	Timing measured	Baseline
Tangi et al. [4]	synthetic	no	no	not stated	model families and ensembles
Zhang et al. [2]	logged	no	not stated	not stated	conventional method
Baluch et al. [3]	logged	no	not stated	not stated	LSTM
Wu et al. [6]	not stated	yes (simulated)	not stated	not stated	Q-learning, DP
Sun et al. [7]	not stated	yes (simulated)	not stated	not stated	SAC, DDPG
Xiong et al. [9]	not stated	yes (HIL)	real-time target (xPC)	not stated	rules, DP
This work	logged	yes (PIL)	yes	yes	rule-based

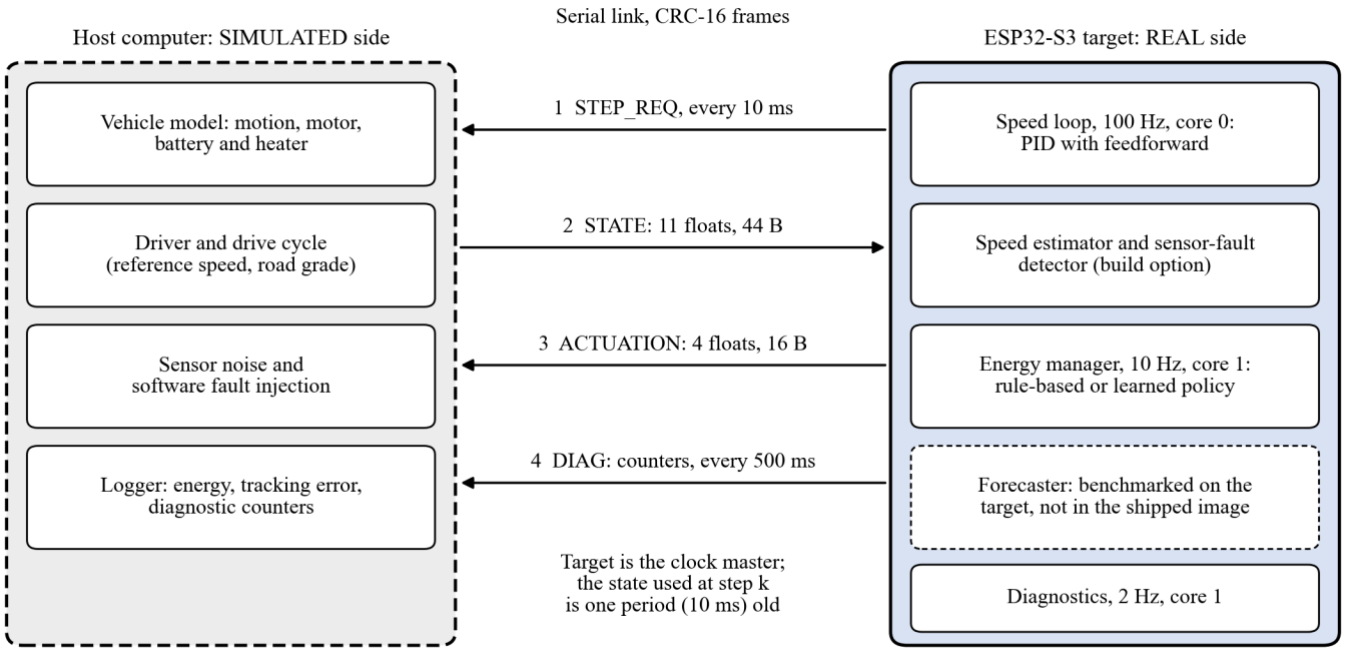


FIGURE 1. Processor-in-the-loop arrangement. The host side is simulated; the target side is real. The forecaster was benchmarked on the target but is not part of the shipped image.

B. TARGET HARDWARE AND FIRMWARE

The target is an ESP32-S3 with two Xtensa LX7 cores at 240 MHz and 8 MB of in-package PSRAM on an N8R8 development board; in the stand-alone compute benchmark, 344,268 bytes of internal heap were free at start-up. Firmware runs under FreeRTOS with four tasks. The speed loop runs at 100 Hz on core 0 at the highest priority; it combines a PID controller with a physics feedforward and, in some builds, a speed estimator and sensor-fault detector. Core 1 hosts an energy-management task at 10 Hz, an inference task at 10 Hz and a diagnostic task at 2 Hz, so that inference cannot delay control. A task watchdog guards the loops.

Which energy manager runs is a compile-time choice. The firmware builds the learned policy when its generated weight header exists and the rule-based manager otherwise; no frame can switch it. A campaign therefore needs one image per controller, and the choice is confirmed through the diagnostic counter that records how many outer steps the policy answered. That counter cannot be non-zero in the rule-based image.

Frames carry a sequence number, a microsecond timestamp and a 16-bit cyclic redundancy check. The diagnostic frame returns counters for late control steps, stale steps, reply timeouts, policy steps and worst-case execution times, with a 13-bin histogram of period deviation.

The firmware was first built with the vendor framework and later moved to the Arduino core, which produces a different binary from the same source. Every gate was measured again for that reason. Integer multiply-accumulate throughput on the Arduino core was 29.82 million per second. The forecaster benchmark fell from 36.46 to 35.03 ms, the policy rose from 0.121 to 0.123 ms, and the 99th-percentile period deviation stayed at 25 µs with no late control step on either build. The Arduino image contains radio stacks that cannot be removed. Nothing starts them, but supply current was not re-measured on this image.

C. ACCEPTANCE GATES

Five gates were written, with pass criteria and an action on failure, before the experiment each one governs. Table 2 lists each gate with its result as recorded. The plant criterion was revised once, after its original pooled form failed on all seven validation trips: it was split into three parts, both original thresholds were kept, and one new threshold was added (Section IV). No other criterion was altered.

In Table 2, RMSE is root-mean-square error and SOC is state of charge. G1 and G5 are measured on the target. G2 is simulated against logged trips, with the failing trip missing the SOC bound by 0.45 points (Section IV). G4 is scored offline on validation data (Section V). The G5 figure times the deployed graph in scalar integer code with fixed weights; it does not run a converted trained model.

D. REAL-TIME BEHAVIOUR

G3 was measured over ten minutes, 59,650 control loops, with the target keeping time against its own clock. On the first Arduino-core run every loop landed within 25 µs of its slot, the worst control computation took 61 µs and no loop was late against its own deadline. Fig. 2 shows the distribution: all counts sit in the first bin, and the tolerance line lies far to the right of them.

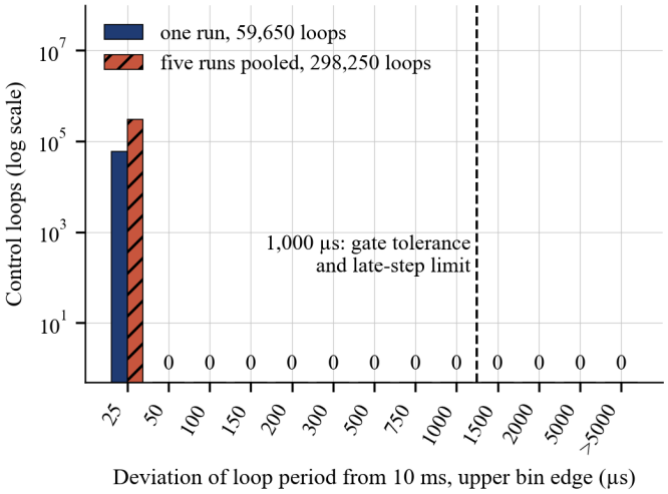


FIGURE 2. Deviation of the control period from 10 ms on the Arduino-core build (measured). Empty bins are marked 0. The five pooled runs use the shipped image with the learned policy.

The frame criterion fails. A dropped frame is counted whenever a reply does not arrive within the 1 ms read window, and the first run had 9. The link carried every frame intact: 121,208 frames were received with none malformed or lost, and the target counted no malformed frame. The record attributes the misses to the host, a general-purpose operating system answering late in short bursts. In the earlier synchronous loop the same cause produced 6 reply timeouts, with a worst round trip of 7,405 µs. A missed reply now means one step on a state one period older than intended, and torque is ramped down only when more than five consecutive steps are stale; this happened once in the first run.

The shipped image was repeated five times. Dropped frames per run were 16, 4, 6, 9 and 9, which is 44 in 298,250 loops, or 8.8 per run with a Poisson 95% interval of 6.4 to 11.8; the events cluster, so the true interval is wider. Late control steps were 0 in every run, the 99th-percentile deviation was 25 µs in every run and the policy answered 5,965 of 5,965 outer steps each time. The criterion was not loosened, and zero drops over ten minutes against a non-real-time host may not be reachable.

E. PROVENANCE AND REPETITION

Each result is tied to the file that produced it and to a label. Campaign cells carry fixed noise seeds: in the 36-run campaign, seed 0, 1 or 2 is paired with the three levels of each scenario axis, and both controllers receive the same draw. The plant and both controllers are deterministic, so without a seeded noise term the three seeds would return identical numbers. For the same reason a repeat with the same seed is not a repeat of anything driven by the noise draw; it replays the draw and varies mainly link timing. Repeats of the fault campaign add seed offsets of 100 and 200.

Anything written twice, in Python and in C, is checked by a parity test that drives both implementations through the same inputs and compares them step for step. This covers the frame format, the order of state fields, the speed controller, the estimator, the fault detector and the decoding of policy actions. The deployed policy matches its NumPy original to 2.12e-06 on the Arduino build with an identical weight checksum.

TABLE 2. Acceptance gates, criteria fixed in advance and results as recorded.

Gate	Question	Criterion fixed in advance	Result
G1	Is the compute budget feasible?	0.1 M multiply-accumulates in 20 ms or less on core 1	Pass: 3.30 ms from measured throughput, margin 6.1x
G2	Does the plant reproduce held-out trips?	Battery-power RMSE within 10% of mean power; final SOC within 2 points	Fails on 1 of 7 validation trips
G3	Does the 10 ms loop close on the target?	99th-percentile jitter at most 1 ms; zero dropped frames in 10 min	Timing passes (25 µs); frame criterion fails
G4	Does a model beat persistence?	At least 10% lower RMSE at 3 s on validation	Pass: weakest fitted model 14.5%
G5	Does the forecaster fit on the target?	Within 40 ms; model plus arena at most 300 KB	36.46 ms (vendor build), 35.03 ms (Arduino core); 28,894 B

The host machine is kept idle during a campaign. Four rule-based cells first run beside other jobs recorded 21 to 54 dropped frames, while late control steps stayed at 0. Cells above 16 dropped frames are set aside and run again. In the later fault campaign, 8 cells were set aside for this reason.

IV. VEHICLE MODEL AND DATA

Every controller in this study acts on a simulated vehicle, so an error in that vehicle is an error in every comparison that follows.

A. DATA

The model was identified and validated on a public archive of 70 trips of a BMW i3 [13]. Each trip logs speed, battery voltage and current, state of charge (SOC), temperatures, elevation and heater power every 0.1 s. Eligibility tests were fixed before any fit: SOC present, no charging, and the two heater-power channels present and in agreement. Of the 70 trips, 33 passed; the summer trips failed, so the validated envelope is winter only, with mean ambient temperature from −3.3 to 10.2 °C.

The 33 trips were split by trip into 19 for identification, 7 for validation and 7 sealed for a final test that has not been opened. Consecutive samples of one drive are near copies, so a row split would put the same moment on both sides.

The Vehicle Energy Dataset (VED) [14] served for a transfer check. Its battery-electric subset is three 2013 Nissan Leaf vehicles in Ann Arbor, with 504 trips. The controller campaigns use the dynamometer schedules UDDS (1369 s, 11.99 km), HWFET (765 s, 16.51 km) and US06 (600 s, 12.89 km) [15].

B. LONGITUDINAL MODEL

The force balance (1) sets the acceleration.

$$m(1+e)\dot{v} = F_t - mgc_{rr}\cos\theta - \tfrac{1}{2}\rho c_d A v^2 - mg\sin\theta \tag{1}$$

Here m is the mass, e the rotational mass factor, v the speed, F_t the wheel force, c_{rr} and c_d the rolling and drag coefficients, A the frontal area, ρ the air density and θ the grade. The wheel force follows from the motor torque T through the gear ratio g_r, wheel radius r_w and drivetrain efficiency η_d.

$$P_e = T\omega/\eta(T,\omega)\ \ (T\omega>0), \qquad P_e = T\omega\,\eta(T,\omega)\ \ (T\omega<0) \tag{2}$$

The electrical power P_e uses the motor speed $\omega = v g_r / r_w$ and an efficiency η from a map over torque and speed, identified on training trips only.

$$V_t = V_{oc}(s) - iR_0 - v_1, \qquad \dot{s} = -\frac{i}{3600\,C_b}, \qquad \dot{v}_1 = \frac{i}{C_1} - \frac{v_1}{R_1 C_1} \tag{3}$$

The battery is a first-order equivalent circuit with terminal voltage V_t, open-circuit voltage V_{oc} at SOC s, discharge current i, series resistance R_0, which the code scales with temperature by −0.4% per °C about 25 °C, branch voltage v_1 across R_1C_1 and capacity C_b. The open-circuit curve, branch values and current limits are assumptions.

$$P_a = P_0 + k_h \max(0, T_s - T_a) + k_c \max(0, T_a - T_s) \tag{4}$$

The auxiliary load P_a depends on ambient temperature T_a about a 21 °C setpoint T_s, with base load P_0 (543 W, measured at rest) and slopes k_h (127.3 W/°C, fitted) and k_c. The cooling slope is unidentified, since every eligible trip is colder than the setpoint.

$$P_r = \max(b\,P_{br},\ -\min(P_{r,max},\ I_c(V_{oc} - v_1))) \tag{5}$$

Regenerative braking returns a fraction b of the braking power P_{br} (negative), limited by an assumed ceiling $P_{r,max}$ of 50 kW and the charge current I_c; friction brakes take the rest. Table 3 lists the main parameters.

TABLE 3. Main parameters of the i3 model, with the origin of each value.

Parameter	Value	Origin
Mass m	1270 kg	published kerb mass plus driver
Frontal area A	2.38 m²	from published drag area and an assumed drag coefficient
Wheel radius, gear ratio	0.3498 m, 9.665	from tyre size; published
Motor limits	250 Nm, 125 kW	published
Drivetrain efficiency η_d	0.95	assumed
Rolling coefficient c_{rr}	0.01947	identified, training trips
Drag coefficient c_d	0.2202	identified, training trips
Rotational factor e	0.1706	identified, training trips
Resistance R_0	0.010 ohm	identified, at lower bound
Capacity C_b	61.7 Ah	Coulomb counting of logged data

C. VALIDATION AGAINST LOGGED TRIPS

Gate G2 was written before the fit. It replays a held-out trip's logged speed and passes only if every validation trip meets the bounds. The original single criterion pooled three models; applied literally it failed all seven trips and could not say which model was at fault. The one revision the design allowed split it into three parts and kept both original thresholds. Part A: torque inverted from the speed profile within 50% of the logged-torque standard deviation, the only new number. Part B: battery power, driven by logged torque, within 10% of mean absolute power. Part C: terminal SOC within 2 percentage points (pp).

The gate fails on Part C. Parts A and B pass on 7 of 7 trips (worst case 48.6% and 8.3%, medians 27.4% and 7.0%), and Part C passes on 6 of 7, with a median of 0.19 pp. Trip B08 misses the SOC bound, 2.45 pp against 2.00 pp (Fig. 3); the

bound was kept and the miss recorded. Two other trips sit near it at 1.86 and 1.76 pp. The instantaneous battery-power error of the original pooled criterion correlates at −0.85 with a trip's torque spread, which points to noise from differentiating a 10 Hz speed signal.

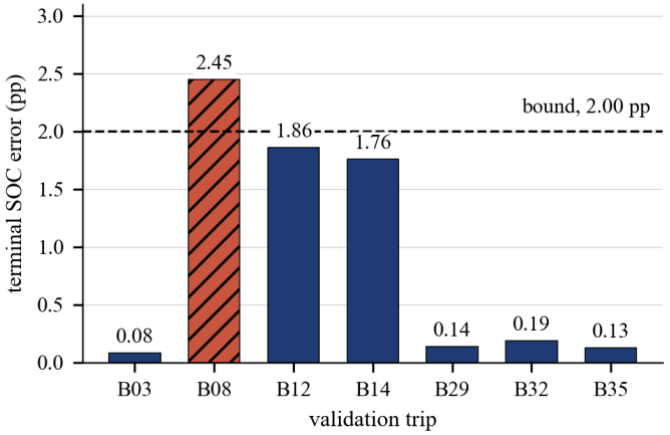


FIGURE 3. Terminal SOC error on the seven validation trips against the 2.00 pp bound of gate G2. Trip B08 (hatched) exceeds it. Replay on the host (simulated against logged trips).

B08 over-predicts energy by 10.4%, and its inverted torque is biased by 13.6% of mechanical energy. Across 26 trips this road-load bias runs from −21% to +30% with no trend against speed, distance or temperature, so one fixed parameter set cannot follow wind, tyre pressure and load. B08 sits in the tail of that spread.

The fitted rolling coefficient is 0.01947, while a published coastdown test of a different battery-electric car, a 2013 Nissan Leaf, gives 0.00878 [33], a ratio of 2.2. A factor of that size between two passenger cars is unlikely to be tyre rolling resistance alone, so the fitted value probably absorbs drivetrain and brake drag, efficiency-map error and unmodelled load. The same holds for c_d and e: all are lumped loss coefficients that reproduce logged power, and none measures the quantity it is named after.

D. TRANSFER TO OTHER VEHICLES

The model was rebuilt around a Leaf and run on 89 VED validation trips (443 of 504 trips were eligible; the 89 VED test trips remain unopened). Arm A carried every i3 parameter across, arm B replaced only the chassis coefficients with the Leaf's coastdown values, and arm C refitted the chassis on VED as the in-domain ceiling. Median trip energy error was 16.9%, 14.4% and 13.4%; 55, 64 and 67 of 89 trips ended within 2 pp of logged SOC. Carrying the i3 coefficients across costs 3.5 points of energy error; the Leaf's measured road load recovers 2.5 of them (simulated against logged trips).

The efficiency map carries over and the chassis coefficients do not, as the 2.2 ratio predicts. The check does not show transfer to electric vehicles in general: it covers three cars of one model in one city, and instantaneous power could not be scored because the noise floor is 77.7% of mean absolute power. Energy error also slopes with temperature in every arm, including arm C, from −15.8% below freezing to +15.6% above 24 °C.

E. A FAULT IN THE POWER LIMIT AND ITS CORRECTION

Each energy manager sends the model a ceiling P_{lim} on battery discharge power. The model applied it only to the bill, $P_b = \min(P_m + P_a, P_{lim})$ with P_m the machine power, and computed acceleration from the torque requested before the cap. A controller that lowered the ceiling was charged less and still received the motion it asked for.

Fig. 4 shows one fixed ceiling held over US06 before and after the fix (simulated on the host). Before, a 5 kW ceiling drove all 12.888 km at 8.05 Wh/km with the same 0.0013 m/s tracking error as an open ceiling at 190.39 Wh/km. After, the same ceiling gives 156.16 Wh/km over 4.453 km with 16.15 m/s tracking error, and a 40 kW ceiling gives 189.80 Wh/km against 174.85 before.

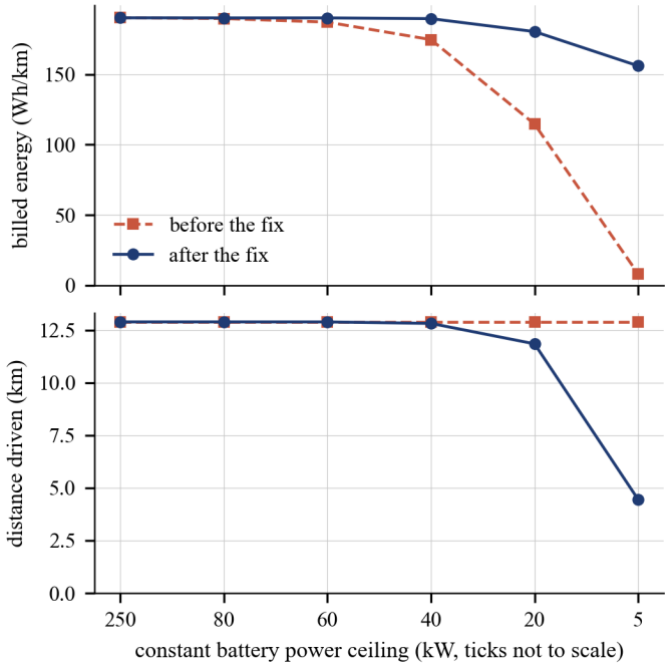


FIGURE 4. A constant battery power ceiling held over US06, before and after the fix. Before, a lower ceiling cut the billed energy and left the distance unchanged; after, it also cuts the distance. simulated on the host.

The fault was found from three oddities in the campaign tables. The learned policy finished below the offline optimum at equal tracking, which should not be possible; its advantage grew with demand; and it vanished on UDDS and HWFET, where demand stays under 40 kW. An earlier explanation, that the policy saved energy by shaving current peaks, had been

recorded as inferred and was wrong. A fixed-ceiling sweep with no controller in the loop reproduced the effect.

The fix gives the motor the power the pack delivered less the auxiliary load, reduces torque to match, and computes acceleration from it. A limit that does not bind leaves the outputs unchanged: in the sweep, the arms whose ceiling never binds give the same energy before and after the fix to the last recorded digit. The rule-based energy manager, whose ceiling rarely binds, should not move. On the microcontroller, across 16 valid settings, the corrected rule arm differed from the earlier campaign by +0.07% on average (95% interval −0.03 to +0.17%), 0.46% at worst, against 0.57% at worst between two campaigns that differed only in the day (measured). Its consequence for the comparison with the learned policy is given in Section VI.

V. ON-DEVICE POWER FORECASTING

A forecast of battery power for the next few seconds has to run on the same microcontroller as the control loop. Two things are reported here: how accurate such a forecast can be, and what each model costs on the ESP32-S3. Accuracy numbers come from validation trips and were computed on the host; timings marked as measured were taken on the target. The test trips were not opened.

A. TASK AND SAFEGUARDS

The task is to predict battery power P_b at each of the next 30 samples, a horizon of 3.0 s at 10 Hz, from a window of the preceding 32 samples (3.2 s). Each sample holds eight signals: vehicle speed, acceleration, state of charge (SOC), battery current, battery voltage, ambient temperature, road grade and auxiliary power. Splits are by trip, with 55,125 training windows from 19 trips and 29,252 validation windows from 7 trips, and the check is made on the group label carried by each window.

Three safeguards were fixed as design requirements before any model was fitted. The first is a check that the target cannot be computed from the inputs. Battery power is the product of voltage and current, and both are inputs, so the window geometry has to leave a gap: inputs end at sample 32 and targets begin at sample 33, and an automated test asserts this on the sample index. The second is a shuffled-label check. A ridge regression with 257 parameters scored an R^2 of 0.4018 on the real targets and -0.0044 on shuffled targets, which is chance. The third is a persistence baseline, which predicts that power stays at its last value. Persistence reaches 13.219 kW of root-mean-square error (RMSE) at 3 s. Extrapolating the trend is worse, at 26.172 kW. Every model had to beat persistence, and gate G4 sets the bar at 10% below it, or 11.897 kW.

B. MODEL LADDER

The ladder runs from persistence to linear regression, gradient-boosted trees (XGBoost), a one-layer long short-term memory network (LSTM) with 64 hidden units, and a Transformer encoder [17] with two layers, four heads and a model width of 32. Table 4 gives the result at 3 s. An untimed latency is an estimate: the multiply-accumulate count divided by the 29.82 million operations per second measured on the target. Each neural model is scored at its best validation epoch, so the validation trips also chose the stopping point and these errors are slightly optimistic; the sealed test trips were not opened to correct for it. The LSTM and Transformer errors are means over training runs of single models, which is what can be deployed; averaging five LSTMs gives 11.007 kW at five times the cost and is left out.

Gate G4 is passed by every fitted model, by at least 14.5%. It was passed before the larger models existed: the linear regression had already cleared it when the baselines were computed. A pass of this gate therefore shows that a forecast is possible but says nothing in favour of a Transformer, and every model is reported against the linear regression as well. XGBoost does not fit the 300 KB memory budget, being about twelve times over it, and remains in the table as an offline reference.

The central comparison is in Fig. 5. The Transformer reaches 11.029 kW against 13.219 kW for persistence, a gain of 16.6%, while linear regression reaches 11.308 kW. Linear regression gives 87% of the Transformer's benefit for 1.2% of its estimated compute. The ordering of the top three is narrow,

TABLE 4. Forecast error at 3 s on validation trips, with the cost of each model on the ESP32-S3. Latency is measured on the target where stated and otherwise an estimate from operation counts.

Model	Error at 3 s (kW)	Gain over persistence (%)	Parameters or size (KB)	Latency on target (ms)	Share of 40 ms budget (%)
Persistence	13.219	0	0	0	0
Linear regression	11.308	14.5	7.5	0.26 (estimated)	0.6 (estimated)
XGBoost	11.290	14.6	about 3,658 (estimated)	not applicable	not applicable
LSTM	11.120	15.9	20.4	35.07 (measured)	88
Transformer	11.029	16.6	17.9	36.46 (measured)	91

since seed spread within a model, 0.038 to 0.174 kW, is the same order as the gaps between models. Fig. 6 shows the error along the horizon. Linear regression and the Transformer stay close at every horizon, and both stay clear of persistence.

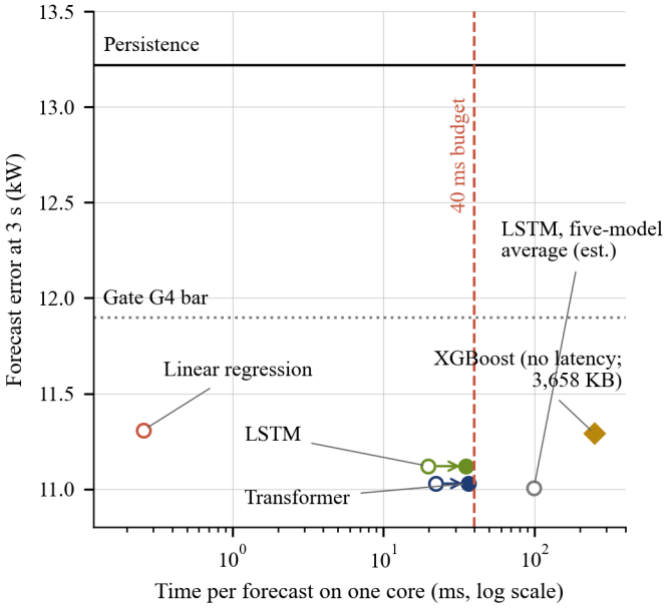


FIGURE 5. Forecast error at 3 s against time per forecast. Open markers are estimates from operation counts; filled markers are measured on the ESP32-S3. XGBoost has no latency figure and is pinned at the right edge.

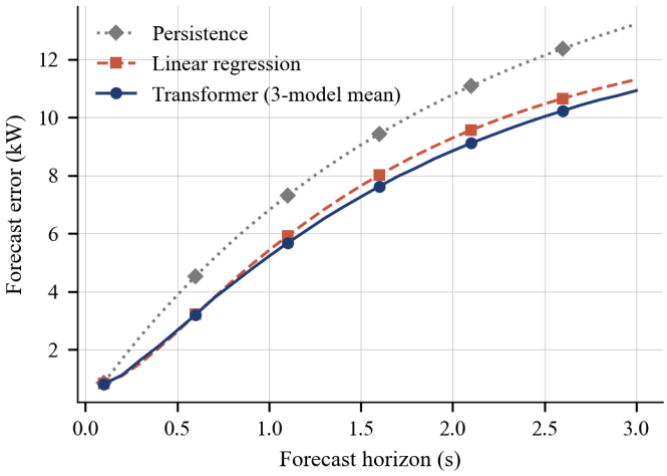


FIGURE 6. Forecast error against horizon on the validation trips, simulated on the host. The Transformer curve is the mean prediction of three trained models and reaches 10.926 kW at 3 s, against 11.029 kW for a single model.

C. PHYSICS-INFORMED TRAINING

A physics-informed variant adds penalty terms to the data loss [18]. The model therefore predicts power, current, voltage and charge, and two residuals tie them together: power must equal voltage times current, and terminal voltage must equal the open-circuit voltage at the predicted charge less the drop across the internal resistance. Each residual is scaled by its channel and weighted by 0.1, chosen on validation RMSE. Plain and informed models share 21,336 parameters and seeds, so only the loss differs.

Accuracy does not change: the mean error over three runs is 11.057 kW for the plain model and 11.052 kW for the informed one. Consistency with the electrical relation does, and by a wide margin. The share of predicted steps in which power disagrees with voltage times current by more than 500 W falls from 24.5% to 2.5%, with the mean disagreement dropping from 373 W to 139 W. The voltage disagreement with the battery relation falls less, from 8.40 V to 5.00 V, which is still above the 2 V tolerance; 72.6% of steps violate it against 91.3% before. The physics terms buy consistency and leave accuracy where it was.

A third relation, charge conservation, had to be switched off. The logged charge channel holds its value across 95.8% of consecutive samples, so the median 3 s window holds one distinct value, and the recorded change is zero against a physically implied 0.00022. Switching the constraint on cost 4.4% of accuracy (11.542 kW) and moved its own violation rate only from 0.988 to 0.966. Charge conservation is testable over minutes, as the trip-energy check of gate G2 does, but not over 3 s on this channel.

D. COST ON THE TARGET

The first cost figure for the Transformer, 22.28 ms, was an estimate: the multiply-accumulate count divided by the measured throughput. The same network was then written in C with 8-bit arithmetic and timed on the ESP32-S3, where one forward pass took 36.46 ms on the vendor-framework build (measured), or 91% of the 40 ms budget of gate G5, and 35.03 ms after the move to the Arduino core (Section III). The measurement was 64% above the estimate. The record attributes part of the gap to work that a multiply count does not price: the softmax needs 1,024 exponentials for each of four heads in each of two layers, which takes 7.25 ms, and layer normalisation adds 0.64 ms; together these explain 7.89 ms of the 14.18 ms difference. The LSTM has no softmax and was further off, 19.84 ms estimated against 35.07 ms measured, so the optimism lies in the estimating method, whichever architecture it is applied to. The timed code is scalar C with fixed weights, so it times the graph and says nothing about what a trained model would predict.

Integer quantisation [25] was assessed separately, on the host. Rounding all weights and activations to 8 bits raised the error of the final-epoch model from 11.051 kW to 11.156 kW, a loss of 0.95%, and reduced the stored model, weights with their scale factors, from 71.7 KB to 20.2 KB, which is 3.6 times smaller; the 17.9 KB in Table 4 counts the weights alone

at one byte each. The timed graph needs 17,600 bytes of weights and 11,294 bytes of working memory, 28,894 bytes or 9.4% of the 300 KB budget. Gate G5 is not closed in full, since the converted model has not been run through an embedded runtime on the target.

E. DISTILLATION

Whether a larger teacher distilled into the deployable student [26] improves it was tested with a teacher of width 64 and four layers, 136,414 parameters, and students of width 32 (18,366 parameters) and width 16 (5,102). Students trained on an equal mixture of the true target and the teacher's output, and arms were compared seed by seed after quantisation. Distillation did not help. The width-32 student was 21.9 W worse (0.20%) and the width-16 student 99.2 W worse (0.89%), and the sign changed between seeds for both.

The record's reason is that the teacher had nothing to teach. It has 26.7 times the parameters of the smallest student and 23.1 times its memory, yet its error after quantisation, 11.212 kW, is 64.3 W worse than that student's 11.148 kW. All five models lie within 99.2 W of each other, 0.89%, which is less than the 0.95% that 8-bit conversion costs. The half-width student needs 6.1 KB and an estimated 6.75 ms, a figure not to be trusted until timed on the target. Whether a forecast of this accuracy is worth anything to the energy manager is the question taken up in Section VI.

VI. ENERGY MANAGEMENT: RULES AGAINST A LEARNED POLICY

A learned energy manager is compared here with a hand-written one, with both running on the microcontroller and driving the simulated vehicle of Section IV. The learned one does not win.

A. CONTROL STRUCTURE AND THE RULE-BASED MANAGER

Two loops share the vehicle. An inner speed loop runs at 100 Hz on the microcontroller and turns the speed reference into motor torque. A slower supervisory energy manager runs at 10 Hz and sends three numbers: the fraction of braking energy routed to the machine (regeneration blend), a ceiling on battery discharge power, and the fraction of the auxiliary load, mainly cabin heating, to shed. The energy manager cannot change the speed reference, which stops a controller from saving energy by driving slower.

The rule-based energy manager is a pure function of state of charge (SOC) and speed, with no memory. Regeneration is full below SOC 0.85, tapers linearly to zero at 0.94, and fades out below 2 m/s so that stops stay predictable. The discharge ceiling is 125 kW above SOC 0.20 and falls linearly to 40 kW at SOC 0.07. Auxiliary shedding stays at zero above SOC 0.15 and rises to a cap of 30% as the pack empties. The constants were not tuned against any test cycle.

The room for improvement is small. An offline dynamic-programming optimum [19] was computed on the WLTC class 3b cycle and the three dynamometer schedules UDDS, HWFET and US06 [15], and the rule-based manager sits within 0.01% to 0.16% of it: 153.90 against 153.89 Wh/km on the highway cycle (HWFET), 167.93 against 167.67 on the urban cycle (UDDS) and 190.47 against 190.39 on the aggressive cycle (US06). The optimum is a constant instruction: recover as much braking energy as the pack accepts. With speed fixed by the inner loop, the discharge ceiling can only remove power the speed loop needs, and cabin shedding is priced at what it saves. A learned policy therefore has at most 0.16% to gain (simulated).

B. THE LEARNED POLICY

The learned policy is trained with Soft Actor-Critic (SAC) [5]. It observes eight scaled quantities: SOC, speed, acceleration, battery temperature, road grade, tracking error, and the mean and maximum of the next 3 s of reference-implied traction power. It outputs the same three numbers as the rules, mapped affinely from $[-1, 1]$ to the physical ranges. The reward is the negative sum of battery energy, shed auxiliary energy at the price it saves, a tracking-error term and penalties for excess current and low SOC. The deployed actor is 8-64-64-3 with 4,931 parameters. Training used the WLTC3B and UDDS cycles, with the HWFET cycle for validation and US06 sealed for the final campaign. Each arm used 5 seeds and 200,000 steps per seed.

The comparable energy scale charges shed cabin heat back at exactly the energy it saved. Without it, a policy that switches the heater off would look efficient on raw Wh/km while delivering a less comfortable vehicle, and every trained arm here scores lower than the rules on the raw figure and higher on the comparable one. All ranking uses the comparable scale.

Seed selection used validation only. After the model correction described below, five seeds per arm were retrained and scored simulated on HWFET: the no-forecast arm reaches 154.86 ± 0.51 Wh/km (seed standard deviation) against 153.90 for the rules, and the arm with the forecast reaches 155.12 ± 0.82. The no-forecast seed with the lowest validation energy, seed 3 at 154.21 Wh/km, was exported to the target. A policy of this shape costs 0.121 ms per decision on average and 19.3 KB of memory on the ESP32-S3 [12], and its outputs match the host implementation to 2e-06 against a tolerance of 2e-05 (measured on another seed of the same network). Inside the campaign loop the worst policy computation was 0.84 ms.

C. FORECAST ABLATION

The forecast ablation removes the two forecast channels from the observation, holds everything else fixed, and compares paired seeds. On the original vehicle model the difference (with minus without forecast) is +0.240 Wh/km with a 95% interval of −0.669 to +1.228 (simulated). After retraining on the corrected model it is +0.262 Wh/km with an interval of −0.698 to +0.982 (simulated). Both intervals contain zero.

The forecaster of Section V is accurate at a 3 s horizon, and this controller still gains nothing from it. Under a fixed speed profile the best action does not depend on what happens next, which matches the dynamic-programming result above. The forecast is not worth its on-target cost to this controller, so the deployed policy is the one trained without it. A policy trained with the forecast and run with two blank inputs would be one that no training run had seen.

D. PROCESSOR-IN-THE-LOOP CAMPAIGN

The campaign closes the loop in the processor-in-the-loop (PIL) arrangement: the control software runs on the microcontroller at 100 Hz, and the host simulates the vehicle, motor, inverter and battery. Six factors were each swept across three levels, one factor at a time from a US06 baseline: driving cycle (UDDS, HWFET, US06), road grade (+0.03, 0 and −0.03 rad), starting SOC (0.85, 0.55, 0.25), sensor noise (1x, 3x and 10x a stated floor), communication delay (none, 10 ms, 30 ms) and driver aggression (a time scaling of the reference by 1.0, 1.25 and 1.5). Each cell ran once per controller with its own noise seed, giving 36 runs and 2,282,832 control steps, of which 0 were late against the loop's own deadline (measured). Vehicle mass was not an axis of this campaign.

The two delay settings above zero are excluded. The delay is injected by making the host wait before each reply, so at 10 ms on a 10 ms loop the host cannot keep up, and neither controller follows the cycle (tracking error 25.7 to 43.4 m/s). That axis therefore measures how fast the host can answer, which says nothing about tolerance to link delay. The rule, tracking error of 1 m/s or more for either controller, was fixed after those runs were seen; the largest valid error is 0.504 m/s. The remaining 16 settings form the valid set.

Table 5 and Fig. 7 give the result. The learned policy uses less energy than the rule-based manager in 1 of the 16 settings, uphill on US06 at −0.02%, a difference smaller than the 0.16% mean spread between two identical rule-arm campaigns. Averaged over the 16 settings it uses 0.94% more. The largest deficits are downhill (+5.71%) and 10x sensor noise (+4.45%); the other 13 settings lie between 0.03% and 0.82%. On tracking, the learned policy has a root-mean-square error (RMSE) of 0.17 to 0.24 m/s on most US06-based settings, 0.33 m/s at driver aggression 1.25 and 0.504 m/s at 1.5, against 0.010 to 0.098 m/s for the rules.

TABLE 5. Corrected PIL campaign, 16 valid settings, one run per cell per controller, comparable energy scale. measured.

Setting	Cycle	Rules (Wh/km)	Learned (Wh/km)	Difference (%)	Rules RMSE (m/s)	Learned RMSE (m/s)
Urban	UDDS	168.13	169.30	+0.70	0.013	0.016
Highway	HWFET	153.96	154.40	+0.29	0.004	0.004
Mixed	US06	191.41	191.48	+0.03	0.044	0.226
No delay	US06	190.73	191.76	+0.54	0.014	0.230
Uphill 0.03 rad	US06	305.26	305.21	−0.02	0.039	0.237
Flat	US06	190.75	191.43	+0.36	0.015	0.229
Downhill 0.03 rad	US06	79.30	83.83	+5.71	0.010	0.193
SOC 0.85	US06	190.93	191.37	+0.23	0.026	0.233
SOC 0.55	US06	191.00	191.63	+0.33	0.027	0.195
SOC 0.25	US06	190.80	191.20	+0.21	0.021	0.168
Noise 1x	US06	190.75	191.81	+0.55	0.021	0.230
Noise 3x	US06	191.33	192.90	+0.82	0.014	0.218
Noise 10x	US06	201.55	210.53	+4.45	0.055	0.192
Driver 1.0	US06	190.99	191.98	+0.52	0.028	0.228
Driver 1.25	US06	197.16	197.38	+0.11	0.045	0.331
Driver 1.5	US06	204.46	204.75	+0.14	0.098	0.504

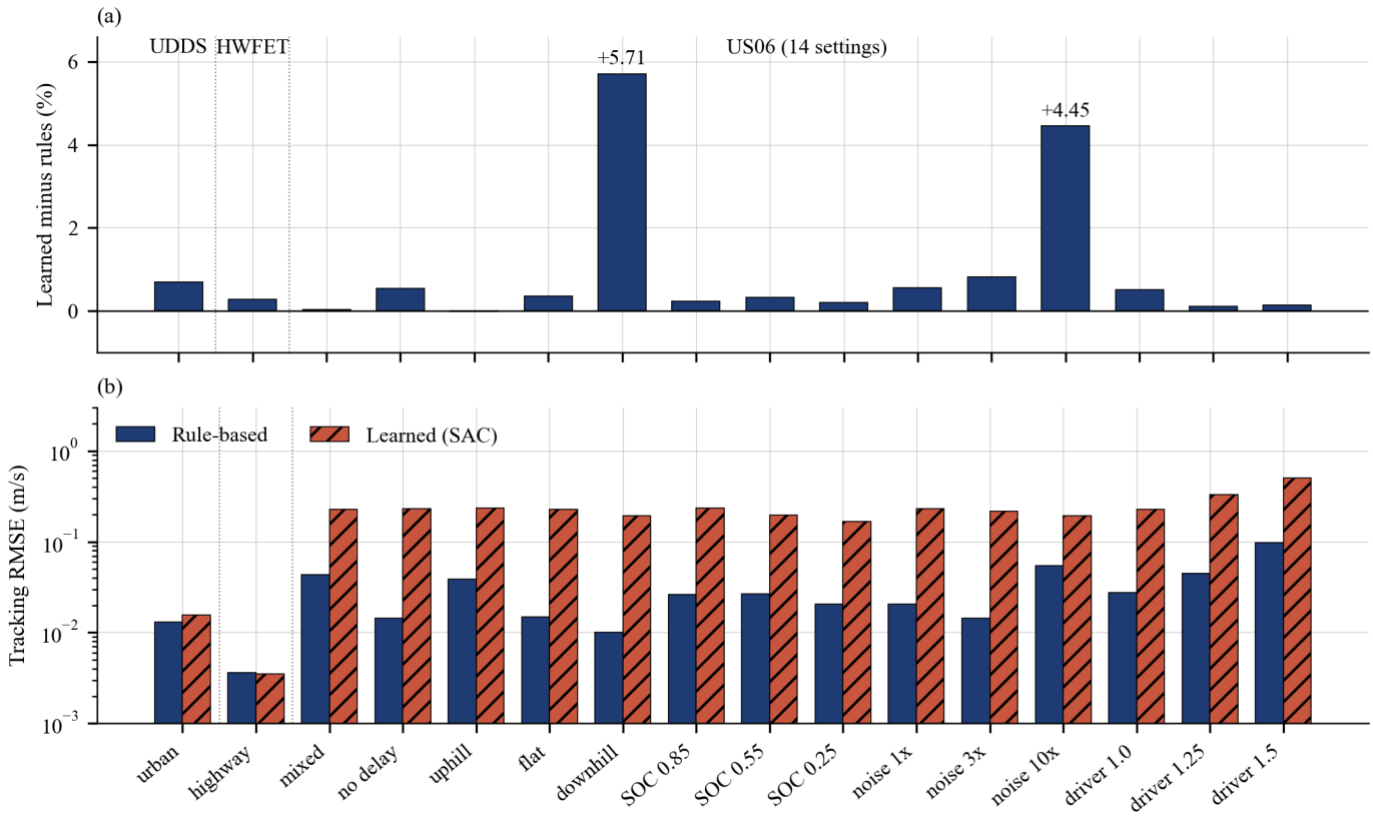


FIGURE 7. Corrected campaign, 16 valid settings: (a) energy difference of the learned policy relative to the rules, positive meaning more energy; (b) tracking RMSE of both controllers on a log axis. measured, one run per cell.

The 16 settings are different conditions, so no interval is attached to their average.

E. AN APPARENT GAIN THAT WAS WITHDRAWN

An earlier campaign, on the vehicle model before correction, appeared to show the opposite. The learned policy used less energy in 14 of the same 16 settings, by 8.28% on average and up to 21.6% at driver aggression 1.5. The gain grew with demand and vanished on UDDS and HWFET. It was an artefact: the model capped the battery power it billed at the policy's ceiling but computed acceleration from the uncapped torque, so a low ceiling reduced the energy charged and left motion untouched (Section IV). The policy had learned to set a low ceiling and still draw power above it, where motion was delivered and not billed. On US06 a constant 5 kW ceiling then billed 8.05 Wh/km for the full 12.89 km (simulated). After the fix and retraining, the gain disappears, while the rule-based arm differed by +0.07% with a 95% interval of −0.03 to +0.17, which contains zero and lies within the spread between identical campaigns. The earlier energy result was withdrawn when the model fault was found, which is why Section IV audits the vehicle model.

F. BEHAVIOUR UNDER LINK OUTAGE

When the host link drops, the firmware reacts in two ways. If the 100 ms energy-manager setpoint is more than 300 ms old, the rule-based action runs inside the speed loop. If no vehicle state has arrived for five control periods (50 ms), the torque command is ramped to zero at 10 N m per step and the integrator is frozen. The fallback ablation compiles these two behaviours out; the firmware then holds the last command.

The sweep used outage lengths of 0.2, 0.5, 1, 2, 3, 4, 5 and 10 s, eight paired onsets on US06 per length, with the retrained policy in the loop and the corrected model (measured). Fig. 8 shows the worst peak speed error. Removing the handling gives the smaller worst error from 0.2 s to 4 s (0.18 against 0.23 m/s at 0.2 s, 3.73 against 6.67 m/s at 4 s). At 5 s the order reverses, 7.40 m/s with the handling against 7.97 without, so the crossover lies between 4 and 5 s; no point inside that interval was measured. At 10 s the worst error is 13.3 m/s with the handling and 18.2 m/s without. The median is less regular: the handling has the lower median at 4 s (1.21 against 1.55 m/s) and at 10 s (3.72 against 5.33 m/s), and the higher one at 5 s (1.64 against 1.44 m/s).

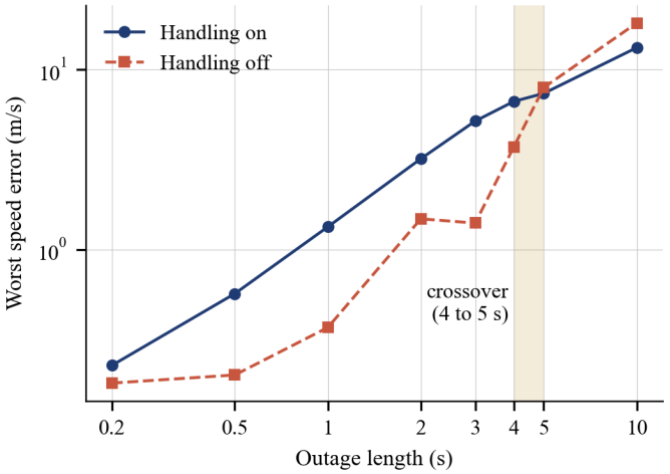


FIGURE 8. Worst peak speed error against outage length, with the fallback handling on and off, on the corrected model. measured, eight onsets per length; lines join measured points only.

The handling is there for safety: it gives up some tracking in short outages so that long ones fail small. Holding a stale command works for a second or two and stops working once the reference has moved on.

VII. SPEED ESTIMATION AND SENSOR-FAULT DETECTION

The positive results of the study concern the speed loop, one level below the energy manager of Section VI. Results are measured when they ran on the ESP32-S3 in the processor-in-the-loop (PIL) arrangement, and simulated when they ran on the host only. Faults were injected in software; no physical accelerometer was used.

A. WHERE THE NOISE LOSS SITS

Speed-sensor noise raises the energy used under the rule-based manager. On US06 the equivalent energy rose from 190.98 Wh/km at the noise floor to 200.42 Wh/km at ten times the floor (measured). The speed loop turns the reading into a torque command at 100 Hz, so noise becomes alternating drive and brake commands. Two readings are possible: the energy manager decides worse on a noisy state, or the speed loop wastes energy on the reversals.

The simulation supports the second reading. With the same manager in every arm, the baseline loop used 200.3 Wh/km at ten times the noise, and replacing only the speed estimate gave 190.6 Wh/km (simulated, five seeds). Torque reversals fell from 1,342 to 83 per km in the same runs. The manager was unchanged, so the loss sits in the speed loop; the loss per reversal was not measured.

B. TWO-STATE KALMAN SPEED ESTIMATOR

The estimator is a Kalman filter [16] over the speed v and an unknown acceleration d, which absorbs what the vehicle model gets wrong, such as extra mass or headwind. The prediction advances the speed with the road-load model a_m of the feedforward and the commanded torque τ. In the pipelined loop of the target the state acted on is one request old, so the prediction must use the command before last; only the target exposed this.

$$\hat{v}_k^- = \hat{v}_{k-1} + [a_m(\hat{v}_{k-1}, \tau_{k-j}) + \hat{d}_{k-1}]\Delta t, \qquad \hat{d}_k^- = \hat{d}_{k-1} \tag{6}$$

Here $j = 1$ in the host loop, $j = 2$ on the target, and Δt is 10 ms. Process allowances of $q = 0.002$ m/s and $q_d = 0.02$ m/s² were set on the first 600 s of the urban training cycle. The measurement z_k corrects both states through the innovation ν_k, with a variance r^2 re-estimated from the jitter of the reading.

$$\nu_k = z_k - \hat{v}_k^-, \quad S_k = P_{vv}^- + r^2, \quad \hat{v}_k = \hat{v}_k^- + \frac{P_{vv}^-}{S_k}\nu_k, \quad \hat{d}_k = \hat{d}_k^- + \frac{P_{vd}^-}{S_k}\nu_k \tag{7}$$

Two firmware images differing only in this input were run on the target, on three cycles at the noise floor, a second seed on US06, and US06 at three and ten times the floor, with noise on every measured channel. Table 6 and Fig. 9 give the result.

TABLE 6. Energy on the comparable scale and peak battery current for the baseline loop and the two-state estimator, measured, one run per cell.

Setting	Baseline, Wh/km	Two-state, Wh/km	Peak current, A
UDDS, floor	168.10	168.05	118 / 111
HWFET, floor	153.98	153.92	97 / 92
US06, floor	190.79	190.61	284 / 262
US06, floor, second seed	190.98	190.56	295 / 260
US06, 3x noise	191.27	190.67	330 / 273
US06, 10x noise	200.42	190.80	341 / 288

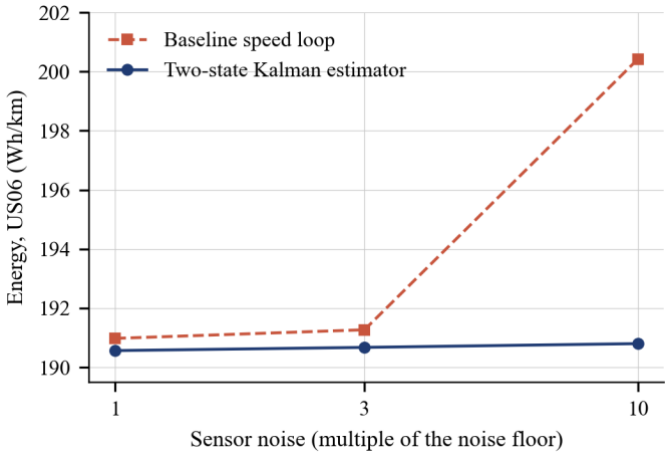


FIGURE 9. Energy on US06 against sensor-noise multiple for the baseline speed loop and the two-state Kalman estimator, measured on the target.

At ten times the noise the baseline loop loses 9.44 Wh/km (4.9%) against its own floor result, and the two-state estimator

loses 0.24 Wh/km. The peak battery current falls from 341 A to 288 A. The worst control computation grows from 62 to 69 µs of a 10,000 µs period, and no step was late. At the noise floor the arms agree within 0.1%, the size of the baseline's own run-to-run difference, so no saving is claimed there.

One pre-registered criterion failed. Clean tracking on the urban cycle had to stay within 0.005 m/s of the baseline, and the estimator missed it by 0.004 m/s (0.0193 against 0.0104 m/s). The baseline's two US06 runs differ by 0.011 m/s with nothing changed, so one run per cell cannot resolve the bound; it stays a failure because the bound came first.

C. FAULT DETECTION FROM AN ACCELERATION RESIDUAL, AND WHY IT FAILED

The first detector compared the speed reading with a speed dead-reckoned from measured acceleration. With an idealised accelerometer, true acceleration plus white noise, it caught 160 of 160 named faults within 0.21 s and raised no false alarm in 3.8 h of clean driving (simulated).

On the target it raised one false alarm on a healthy sensor in 1.26 h (measured). The alarm recurred at the same control step in repeated runs and in a host replay, because each campaign cell used one fixed noise seed. The replay showed that the dead-reckoned speed had drifted 0.018 m/s low while the speed noise averaged 0.013 m/s high, and the threshold allowed for the second error only.

Against 70 logged trips (30 h, no sensor fault) the logged acceleration and the acceleration implied by logged speed differ by a slow component of about 0.11 m/s². Over the two-second pull of the dead reckoning this leaves a standing gap of about 0.22 m/s against a threshold of 0.036 m/s, and the detector was flagged 99% of the time. The logs are 10 Hz and the firmware 100 Hz, so the flagged share is indicative. The detector is not usable as tuned.

D. BIAS-AWARE FAULT DETECTOR

The bias-aware detector keeps the filter above and changes the reference it is tested against. The dead-reckoned speed carries a state b for the acceleration-channel error, learned with a 2 s time constant and subtracted before integration, so a standing offset opens no gap. A step test handles abrupt faults: a sensor offset arrives in one sample and channel wander does not, so the mean gap $\bar{g}$ of the last $h = 30$ samples is compared with that of the 30 before, and the slow error common to both cancels.

$$|\bar{g}_{\text{new}} - \bar{g}_{\text{old}}| > t_s \sqrt{\frac{2\max(s^2, r^2)}{h}} + \alpha h \Delta t \tag{8}$$

Here s^2 is the pooled sample variance of the gap, $t_s = 6$ and $\alpha = 0.2$ m/s² allows for a changing channel error. The 20-sample mean test remains, with the dead reckoning's own standard error of 0.15 m/s added to its threshold. The step threshold was tuned on the training cycles only, and the registered cases and criteria were not altered after the result was seen.

The registered cases combine five named faults with six acceleration-channel errors and a vehicle 10% heavier than the model. In simulation the detector raised no clean false alarm on validation (HWFET) or test (US06) and caught 551 and 533 of 560 named faults within 0.5 s. It does not pass the registered criteria: the nine late detections on validation are +0.5 m/s offsets at ten times the noise, flagged only when they end, and the 27 late detections on test were not characterised. The earlier detector, on the same cases, was flagged for 75% to 98% of clean validation driving in five of seven.

On the target the host injected the faults and the detector ran in C, matched step by step to the Python version on 4,000 samples. Table 7 and Fig. 10 give the result for US06 (measured).

TABLE 7. Injected faults caught within 0.5 s on the target by fault type (measured). First campaign: 8 runs per type. Three noise draws: 24 runs per type.

Fault	Earlier detector, first campaign	Bias-aware, first campaign	Bias-aware, three draws	Worst tracking error, three draws, m/s
Stuck	7 of 8	8 of 8	24 of 24	0.62
Dropout	7 of 8	8 of 8	24 of 24	0.65
Offset +0.5 m/s	6 of 8	8 of 8	24 of 24	0.83
Offset −0.5 m/s	6 of 8	8 of 8	24 of 24	0.72
Offset +3.0 m/s	7 of 8	8 of 8	24 of 24	0.66
All	33 of 40	40 of 40	120 of 120	0.83

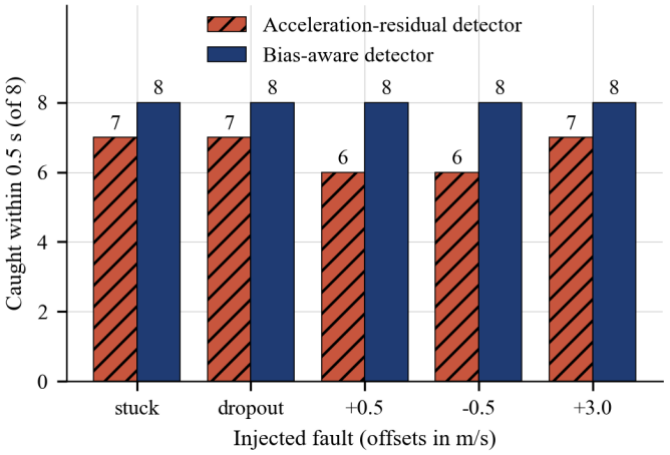


FIGURE 10. Faults caught within 0.5 s in the first campaign on the target, earlier detector against bias-aware detector (measured). Eight faults per type.

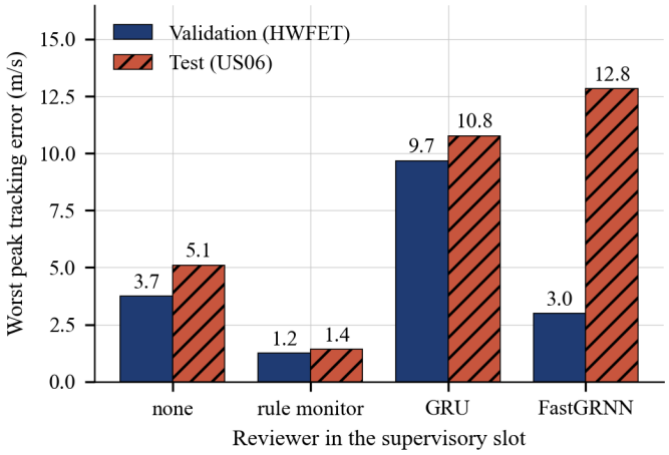


FIGURE 11. Worst peak tracking error through any of 560 named faults for each reviewer on the controller with no delay (simulated). No reviewer means the bias-aware detector alone.

In the first campaign the bias-aware detector caught 40 of 40 faults within 0.32 s. The earlier detector caught 33 of 40 and raised 162 false alarms on one clean cycle under the wandering channel. With three noise draws the bias-aware detector caught 120 of 120, with no false alarm in one clean hour, no late step and a worst control computation of 74 µs.

E. LEARNED REVIEWERS AGAINST A RULE MONITOR

A slower supervisory slot sits around the detector. Every 100 ms it receives twelve summary numbers and may hold a flag, release it, switch the fallback speed, or tighten the tests; it never commands torque. The slot was filled by a hand-written rule monitor, a gated recurrent unit (GRU) [31] of 1,508 parameters and a FastGRNN [32] of 550, on the controller or behind a link with 70 to 1000 ms delay (simulated throughout).

Scored on diagnosis alone over 540,960 validation blocks, the GRU was better: it recognised a real fault in 99% of blocks with a flag raised and 88% with none, against 94% and 1% for the monitor. In the loop the order reversed (Fig. 11). The monitor cut the worst peak tracking error through a fault from 3.7 to 1.2 m/s on validation and from 5.1 to 1.4 m/s on test. The GRU reached 9.7 m/s on validation and 10.8 m/s on test in its worst cases, the FastGRNN 3.0 and 12.8 m/s, and raised clean false alarms. Offline agreement did not predict closed-loop benefit. A correct diagnosis keeps the flag up, and both fallback speeds then drift further than the small offset they guard against.

The target corrected the monitor's result. In the first run it looked slightly better than the detector alone (largest error 0.61 against 0.67 m/s). Over three noise draws it was 1.32 against 0.83 m/s, with three alarms left raised where the detector alone left none (measured). No benefit on the target is claimed.

F. FUSED FALLBACK ESTIMATOR

Both fallback speeds drift, one with the acceleration error and one with the model error. The fused fallback estimator is one filter over speed, model error and accelerometer error that learns both while the sensor is trusted and carries them through a flagged period. In simulation on the validation highway cycle the worst error was 0.62 m/s against 3.74 m/s for the detector alone, with no stuck flag. On US06, against the detector alone and the rule monitor, it was best in six of seven conditions, with two alarms left stuck, and worst in the seventh: with the vehicle 10% heavier than the model the worst error was 2.59 m/s. The cause was not tested; the settings came from gentle urban driving. This estimator has not been ported to C and has not run on the target (simulated only).

VIII. SPEED PLANNING

Everything in this section is simulated on the host computer; nothing ran on the target. The earlier sections gave the controller a speed trace to follow. Here the vehicle chooses the trace. A route is built from a standard drive cycle with the driver removed: it keeps the cycle's distance, its stops and the wait at each, a speed limit at every point, the hardest acceleration and braking the cycle used, and its duration as the arrival time within 1%. The US06 (aggressive) and HWFET (Highway Fuel Economy Test) routes serve as test and validation sets.

Five arms were compared. The cycle as driven is the reference. A classical dynamic-programming (DP) planner [19] searches the whole route in 10 m steps, with a price on time set by bisection so that the arrival time is met. The same planner was also run with a 300 m horizon, replanning every 40 m, and once more with a small network supplying a terminal cost, an estimate of what the rest of the route costs. The fifth arm is a student policy, a network trained on the planner's solutions that names the next speed directly. Every profile was driven through the full vehicle model with the speed loop and the rule-based energy manager, and energy is stated on the comparable scale (Table 8).

TABLE 8. Energy saved against the cycle as driven, by planning arm; one run per route, simulated, no other traffic.

Arm	US06 route (%)	HWFET route (%)
Cycle as driven	0.0	0.0
DP, whole route	10.4	1.2
DP, 300 m horizon	9.5	0.8
300 m, learned terminal cost	10.4	1.1
Student policy	0.0	0.3

On the US06 route the whole-route planner saved 10.4% at the same stops and arrival time; the HWFET route offered 1.2%, since the drive has little to smooth. The terminal cost closed the gap left by the short horizon; the student policy saved nothing on US06.

An empty road is the upper bound on what planning gives. To represent traffic, the planner was held within a corridor around the recorded drive, ahead or behind, at every second, with distance and arrival time unchanged. Fig. 12 shows that the saving on US06 was 5.1% within 25 m and 9.7% within 100 m, while HWFET stayed between −0.1% and 1.7%. The corridor planner differs in construction from the empty-road planner, so its 200 m figure of 12.6% is not comparable with the 10.4% above.

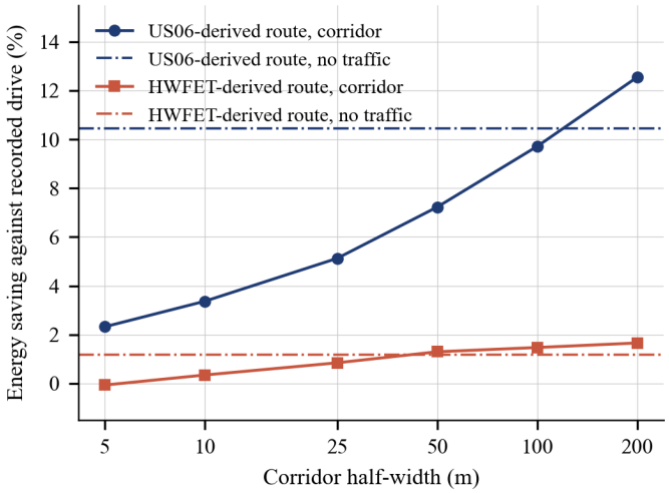


FIGURE 12. Energy saving against corridor half-width, with the no-traffic saving as a dash-dotted reference line for each route. simulated; one run per width.

No other vehicle was simulated, and the corridor around one recorded drive stands in for traffic. Hills and signals are absent, each arm was run once, and the planner has not been ported to the target.

IX. DISCUSSION AND LIMITATIONS

A. WHY THE RULES HELD

The result rests on how little room the problem leaves. The offline optimum, computed by dynamic programming over each whole cycle, was within 0.01% to 0.16% of the rule-based energy manager on WLTC3B, UDDS, HWFET and US06. Recovering braking energy is worth 5% to 23% between the highway and city cycles, and the rules already take all of it. With the speed loop fixed by a tracking constraint, the discharge limit can only remove power the loop needs, and heating priced at its cost cancels out. A learned policy therefore has almost nothing to win and a tracking penalty to pay, which matches the measured outcome (Section VI).

The forecast ablation agrees: after retraining on the corrected model, removing the forecast changed energy by +0.262 Wh/km with a 95% interval of −0.698 to +0.982, which contains zero. Better offline scores did not carry into the loop either. The learned reviewers classified faults better than the rule monitor in shadow scoring, then controlled the vehicle worse in closed loop (simulated, Section VII). These statements hold for this vehicle model, these four cycles and this class of processor, and no further.

B. WHAT THE METHOD CAUGHT

Four findings came from the gates and the repeat rules rather than from the experiments they guarded. The first is the vehicle-model power ceiling. It cut the energy billed without

cutting delivered torque, which produced an apparent 8.28% advantage for the learned policy in 14 of 16 settings. After the model was corrected the policy won 1 of 16 and used 0.94% more energy on average, and the earlier gain is withdrawn here.

The second is the compute estimate for the forecaster, 22.28 ms from a multiply count, against 36.46 ms measured on the target. Measured latency was 64% above the estimate, which is why estimates are used here only to rank models. The third is a false alarm of the acceleration-residual detector, raised once in 1.26 h of clean driving. It returned at the same control step in two repeats and in a host replay, because each setting used one fixed noise seed: a same-seed repeat is not a repeat, and the alarm was one event seen four times. The fourth is the rule monitor. Its largest speed error during a fault was 0.61 m/s in the first run, slightly under the detector alone, but over three noise draws it was 1.32 m/s against 0.83 m/s, so the advantage did not hold on the target.

C. LIMITATIONS

The arrangement is processor-in-the-loop: the processor is real, while the motor, inverter and battery are simulated, so nothing here is a hardware-in-the-loop result. One microcontroller board was used. One vehicle model was identified, and it fails gate G2 on one of seven validation trips, missing the state-of-charge bound by 0.45 percentage points (2.45 against 2.00). Gate G3 fails its zero-dropped-frame criterion, because the host answered late 9 times in the first run of 59,650 loops and 8.8 times per run over five repeats; no control step finished late.

Sensor faults were injected in software and no physical accelerometer was used. The logged acceleration disagrees with the logged speed by a slow component of about 0.11 m/s², which a simulation with random noise alone does not show. The fused fallback estimator, the learned reviewers comparison and the speed planner are simulated only, and the fused estimator fails for a vehicle 10% heavier than its model on US06 (largest error 2.59 m/s). The main comparison has one run per campaign cell. The learned policy is one algorithm, SAC (Soft Actor-Critic), with one reward design; the result does not show that no learned policy could win.

D. FUTURE WORK

A physical accelerometer on a test vehicle would replace the simulated drift, and would show whether the 0.11 m/s² disagreement is a property of the logged data or of the sensor. The fused fallback estimator needs its heavier-vehicle case fixed before it is ported to the target and measured there. The planning task needs a traffic model in place of one recorded drive. The planner itself needs a measured planning time on the processor, because the replanning distance of 40 m sets the time allowed.

X. CONCLUSION

The earlier study [4] left three questions open: whether data-driven models agree with logged trips, whether they can close a control loop, and whether they run on an embedded controller under a real-time deadline. On logged trips, a longitudinal model identified from BMW i3 trips reproduces six of seven validation trips within the bounds of gate G2 and fails on the seventh, with larger errors on the Vehicle Energy Dataset transfer check (median energy error 13% to 17%). In closed loop, a learned policy drove the simulated vehicle. On an ESP32-S3 at 100 Hz, 36 runs covered 2,282,832 control steps with no late step (measured).

After the vehicle-model fault was fixed, the learned policy used less energy than the rule-based energy manager in 1 of 16 settings on the target, 0.94% more on average, and tracked speed worse on US06. The forecast changed nothing measurable. Two positive results were measured on the target: a two-state Kalman speed estimator cut the energy lost to 10x sensor noise from 9.44 to 0.24 Wh/km on US06, and a bias-aware detector caught 120 of 120 injected faults with no false alarm in one clean hour. On this hardware and this problem, classical estimation and hand-written rules survived measurement. The single most useful next step is a physical accelerometer, because the fault results and the fallback estimator all rest on simulated acceleration drift.

REFERENCES

[1] A. Biswas and A. Emadi, "Energy management systems for electrified powertrains: State-of-the-art review and future trends," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 7, pp. 6453–6467, July 2019, doi: 10.1109/TVT.2019.2914457.

[2] J. Zhang, Z. Wang, P. Liu, and Z. Zhang, "Energy consumption analysis and prediction of electric vehicles based on real-world driving data," *Applied Energy*, vol. 275, pp. 115408, October 2020, doi: 10.1016/j.apenergy.2020.115408.

[3] M. B. Baluch, S. M. Sharkh, and A. Madhusudhanan, "Electric vehicle energy consumption modeling using real world driving data: System identification vs. machine learning," *IEEE Transactions on Intelligent Vehicles*, vol. 11, no. 3, pp. 401–414, March 2026, doi: 10.1109/TIV.2026.3656694.

[4] S. Tangi, A. Vatsa, A. Opam, P. K. Bonthagorla, and D. N. Gaonkar, "Smart strategies for improving electric vehicle battery performance and efficiency," *Scientific Reports*, vol. 15, pp. 42070, November 2025, doi: 10.1038/s41598-025-25987-1.

[5] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proceedings of the 35th International Conference on Machine Learning (PMLR)*, vol. 80, pp. 1861–1870, July 2018.

[6] J. Wu, H. He, J. Peng, Y. Li, and Z. Li, "Continuous reinforcement learning of energy management with deep Q network for a power split

hybrid electric bus," *Applied Energy*, vol. 222, pp. 799–811, July 2018, doi: 10.1016/j.apenergy.2018.03.104.

[7] W. Sun, Y. Zou, X. Zhang, N. Guo, B. Zhang, and G. Du, "High robustness energy management strategy of hybrid electric vehicle based on improved soft actor-critic deep reinforcement learning," *Energy*, vol. 258, pp. 124806, November 2022, doi: 10.1016/j.energy.2022.124806.

[8] X. Hu, T. Liu, X. Qi, and M. Barth, "Reinforcement learning for hybrid and plug-in hybrid electric vehicle energy management: Recent advances and prospects," *IEEE Industrial Electronics Magazine*, vol. 13, no. 3, pp. 16–25, September 2019, doi: 10.1109/MIE.2019.2913015.

[9] R. Xiong, Y. Duan, J. Cao, and Q. Yu, "Battery and ultracapacitor in-the-loop approach to validate a real-time power management method for an all-climate electric vehicle," *Applied Energy*, vol. 217, pp. 153–165, May 2018, doi: 10.1016/j.apenergy.2018.02.128.

[10] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, pp. 3207–3214, April 2018, doi: 10.1609/aaai.v32i1.11694.

[11] G. Dulac-Arnold et al., "Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis," *Machine Learning*, vol. 110, no. 9, pp. 2419–2468, April 2021, doi: 10.1007/s10994-021-05961-4.

[12] Espressif Systems, "ESP32-S3 Series Datasheet," *Espressif Systems (datasheet)*.

[13] M. Steinstraeter, J. Buberger, and D. Trifonov, "Battery and Heating Data in Real Driving Cycles," *IEEE DataPort (dataset)*, October 2020, doi: 10.21227/6jr9-5235.

[14] G. Oh, D. J. LeBlanc, and H. Peng, "Vehicle Energy Dataset (VED), a large-scale dataset for vehicle energy consumption research," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 4, pp. 3302–3312, April 2022, doi: 10.1109/TITS.2020.3035596.

[15] U.S. Environmental Protection Agency, "Dynamometer drive schedules," *U.S. EPA, Vehicle and Fuel Emissions Testing (web resource)*.

[16] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, March 1960, doi: 10.1115/1.3662552.

[17] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems 30*, vol. 30, pp. 5998–6008, 2017.

[18] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, February 2019, doi: 10.1016/j.jcp.2018.10.045.

[19] R. Bellman, "Dynamic Programming," *Princeton University Press, Princeton, NJ*, 1957.

[20] N. Kim, S. Cha, and H. Peng, "Optimal control of hybrid electric vehicles based on Pontryagin's minimum principle," *IEEE Transactions on Control Systems Technology*, vol. 19, no. 5, pp. 1279–1287, September 2011, doi: 10.1109/TCST.2010.2061232.

[21] G. Paganelli, S. Delprat, T. M. Guerra, J. Rimaux, and J. J. Santin, "Equivalent consumption minimization strategy for parallel hybrid powertrains," *Proceedings of the IEEE 55th Vehicular Technology Conference (VTC Spring 2002)*, vol. 4, pp. 2076–2081, 2002, doi: 10.1109/VTC.2002.1002989.

[22] A. Sciarretta, M. Back, and L. Guzzella, "Optimal control of parallel hybrid electric vehicles," *IEEE Transactions on Control Systems Technology*, vol. 12, no. 3, pp. 352–363, May 2004, doi: 10.1109/TCST.2004.824312.

[23] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021.

[24] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," *Advances in Neural Information Processing Systems 33*, vol. 33, 2020.

[25] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, June 2018, doi: 10.1109/CVPR.2018.00286.

[26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531 (NIPS 2014 Deep Learning and Representation Learning Workshop)*, March 2015, doi: 10.48550/arXiv.1503.02531.

[27] D. Seo and J. Kang, "Dynamic-model-free vehicle velocity estimation using extended Kalman filter with IMU, steering angle, and wheel speed sensors," *Measurement*, vol. 242, pp. 115810, January 2025, doi: 10.1016/j.measurement.2024.115810.

[28] R. Isermann, "Model-based fault-detection and diagnosis: Status and applications," *Annual Reviews in Control*, vol. 29, no. 1, pp. 71–85, 2005, doi: 10.1016/j.arcontrol.2004.12.002.

[29] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*, pp. 89–94, April 2015.

[30] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, July 2009, doi: 10.1145/1541880.1541882.

[31] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014, doi: 10.3115/v1/D14-1179.

[32] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," *Advances in Neural Information Processing Systems 31*, vol. 31, 2018.

[33] Idaho National Laboratory and Intertek, "2013 Nissan Leaf, advanced vehicle testing, baseline testing results," *U.S. Department of Energy, Vehicle Technologies Program (technical report)*.

MANIKANTA GONUGONDLA [Biography and photograph to be supplied.]